CSE 598 — Spring 2026

τ-Bench Phase 2: Multi-Agent System Proposal

A Multi-Agent Framework for Tool-calling Agent Correction

Group Members

Abhishek Mulasi | Devesh Valluru | Divyesh Patel | Wei-An Wang

Deliverable 3: Multi-Agent System Architecture & Rationale

February 2026

1. Introduction

This proposal presents a multi-agent framework designed to address the systematic failure modes identified in our Phase 2 error analysis of τ-bench baseline agents. Our analysis of 6,282 trajectories across four Qwen3 model sizes (4B, 8B, 14B, 32B) and three baseline strategies (Act, ReAct, Function Calling) revealed that 81% of all trials fail, with a clear taxonomy of recurring error patterns.

The central insight driving our design is that scaling model size does not uniformly reduce errors; it shifts the error profile. Smaller models (4B) predominantly select the wrong tools (43.4% of failures), while larger models (14B, 32B) increasingly hallucinate task completion without executing any tool calls (34–37% of failures). This means a single intervention cannot address all failure modes; instead, different errors require different types of mitigation applied at different stages of the agent pipeline.

This framework addresses this through a five-module pipeline where only two modules require LLM inference (one call at task start, plus the base agent), while three modules are implemented as lightweight, deterministic code-based checks. This design minimizes computational overhead while maximizing error coverage across all identified failure categories.

2. Motivation from Error Analysis

Our custom error taxonomy, developed independently from the τ-bench paper's four-category scheme, identified 8 distinct error types organized into a 5-level classification hierarchy. The figure below shows the aggregate distribution across 5,090 failed trajectories:

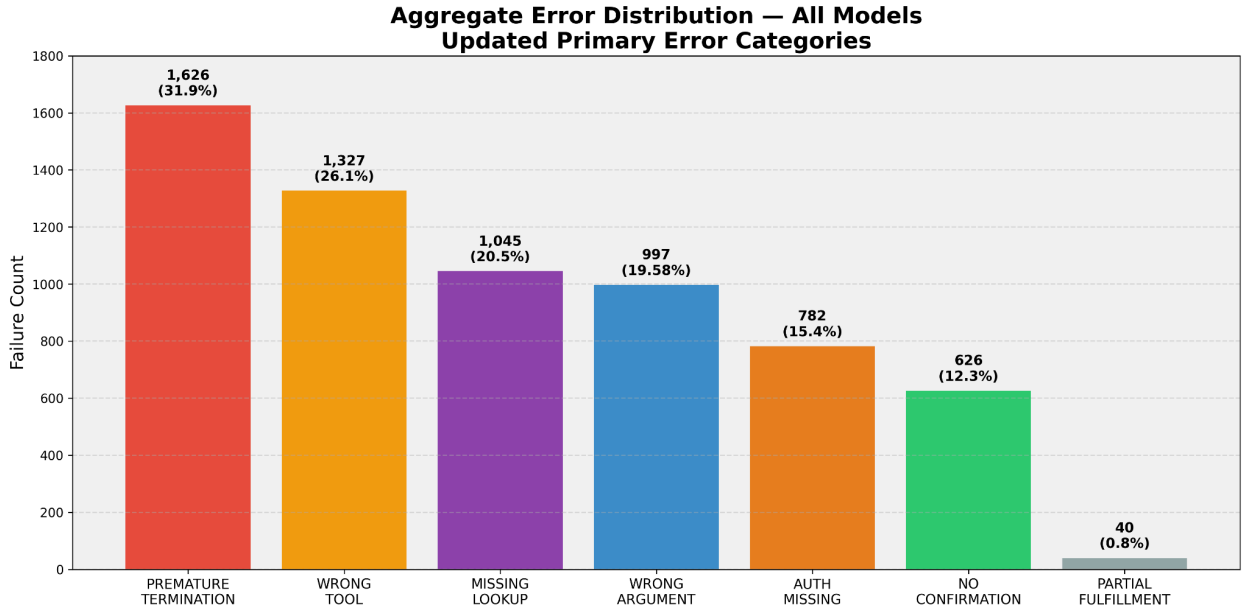

Figure 1: Aggregate error distribution across all models and baselines (excluding EMPTY_TRAJECTORY).

When grouped by root cause, three dominant failure mechanisms emerge:

Cause Category	Error Types	Count	% Failures
Execution Failure	PREMATURE_TERMINATION + EMPTY_TRAJECTORY	2,951	57.9%
Tool Selection Error	WRONG_TOOL	1,327	26.07%
Argument Error	WRONG_ORDER, WRONG_ITEMS, WRONG_REPLACEMENT, WRONG_PAYMENT, WRONG_REASON, WRONG_ARGUMENT, PARTIAL_FULFILLMENT	997	19.58%
Authentication Bypass	AUTH_MISSING	782	15.3%
Incomplete Resolution	MISSING_LOOKUP + PARTIAL_FULFILLMENT	1,085	21.3%
Protocol Violation	NO_CONFIRMATION + WRONG_ARGUMENT	626	12.29%

Table 1: Cause-oriented error grouping with aggregate counts.

2.1 The Error Profile Shift

A critical finding is that model scaling changes, rather than simply reducing error rates, determine which errors dominate. The 4B model's failures are dominated by WRONG_TOOL (43.4%), indicating it cannot select the correct API endpoint. By contrast, the 14B and 32B models largely resolve tool selection but introduce a new failure mode: PREMATURE_TERMINATION rises from 24.6% (4B) to 69.3% (8B), 37.0% (14B), and 35% (32B). Manual inspection of 32B samples revealed that approximately 70% of these cases involve the agent hallucinating task completion — generating convincing confirmation messages (e.g., "Your order has been cancelled, refund of $1,229.82 will be processed") without ever calling any tool.

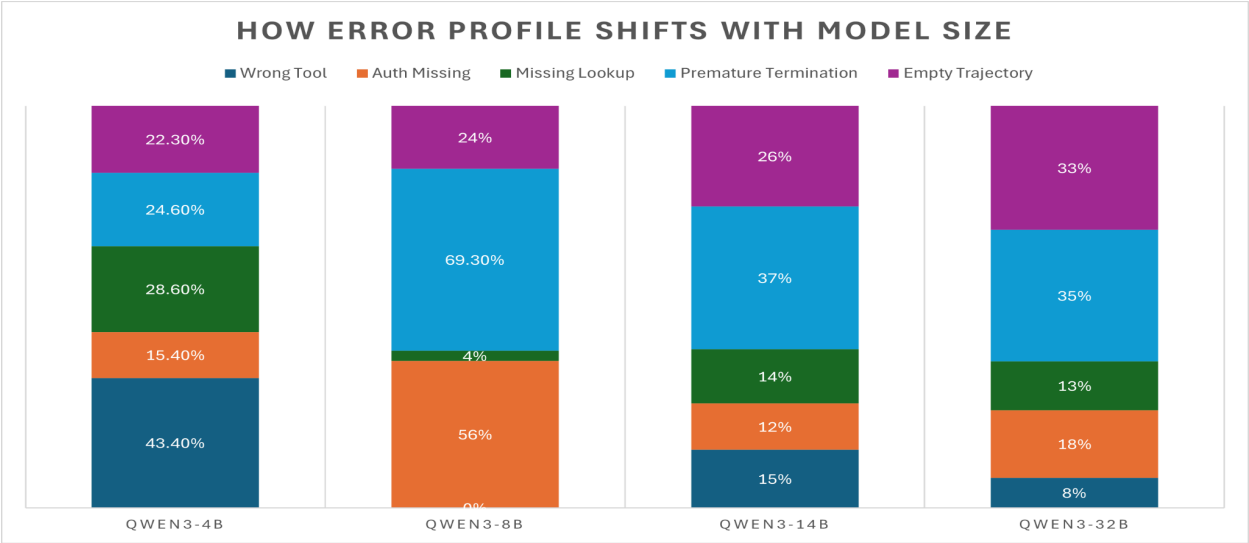

Figure 2: Error profile shift across model sizes showing the transition from tool selection errors (4B) to execution hallucination (8B/14B/32B).

This shift has a direct architectural implication: pre-action interventions (like IRMA's input reformulation) can help smaller models choose the right tools, but they cannot detect post-hoc hallucinations from larger models. Our Framework addresses both failure regimes through a combination of pre-action planning and post-action verification.

3. Related Work

IRMA (Ding et al., 2025) introduces a three-module framework (Memory, Constraints, Tool Suggestion) that reformulates agent inputs to reduce errors in τ-bench. Their ablation study shows that the Constraints module provides the largest individual gain. However, IRMA is explicitly "verification-loop-free", it does not check whether the agent actually executed the tools it claimed to. Our error analysis shows this is the dominant failure mode for larger models (32B), which IRMA's design cannot address.

Agent-R (Yuan et al., 2025) uses Monte Carlo Tree Search to train agents to reflect on and recover from errors via iterative self-training. While powerful, this requires training data generation and model fine-tuning, making it impractical for our setting where we evaluate fixed Qwen3 models without parameter modification.

CORRECT (Yu et al., 2025) proposes a training-free error recognition framework using cached error schemata for multi-agent systems. Their approach of distilling error patterns into lightweight lookup structures aligns with our Action Verifier's design philosophy, though we apply verification at the tool-call level rather than inter-agent communication.

ReIn (Kim et al., 2026) introduces Reasoning Inception, a test-time intervention that injects external reasoning into an agent's decision-making process for error recovery. This test-time, parameter-free approach is similar in spirit to our Task Planner, though ReIn focuses on user-induced conversational errors while we target agent-side tool execution failures.

4. Framework Architecture

Our framework augments the baseline tool-calling agent with four functional modules organized into three phases: pre-action (Task Planner + Context Injector), execution (Tool Agent), and post-action (Action Verifier + Completion Check). The architecture is designed as a single augmented pipeline rather than multiple independent agents, minimizing inference overhead.

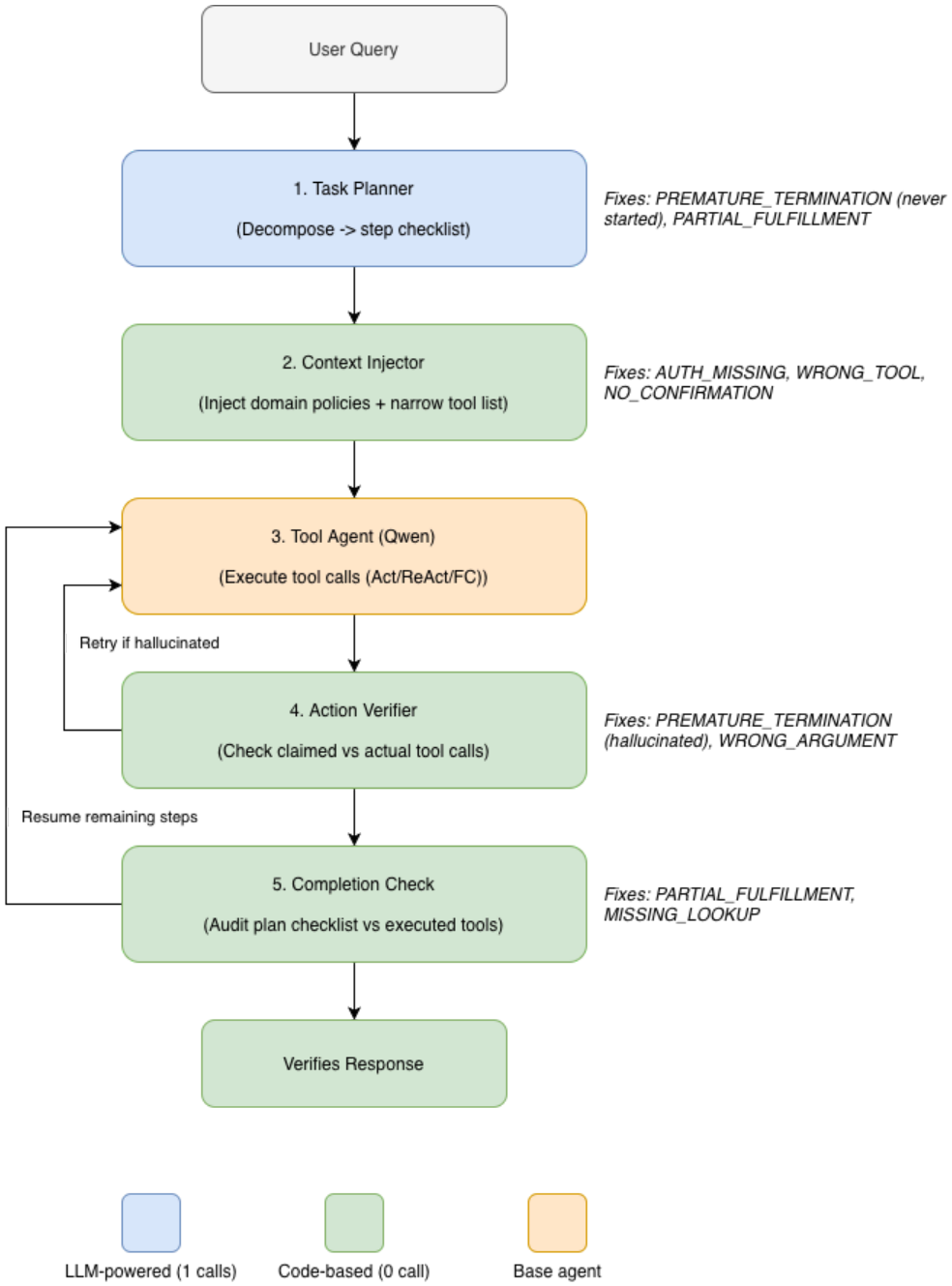

Figure 3: Our framework pipeline architecture. Blue modules are LLM-powered; green modules are code-based. Annotations indicate which error types each module addresses.

4.1 Module 1: Task Planner (LLM, One-time Call)

The Task Planner reads the initial user instruction and decomposes it into a structured step checklist. This module runs exactly once at conversation start, adding one LLM inference call to the total pipeline cost.

Target errors: PREMATURE_TERMINATION (sub-pattern A: agent never started, ~40% of PT cases) and PARTIAL_FULFILLMENT. Our manual analysis found that 40% of premature termination cases involve agents that never initiate tool calls because they fail to extract actionable steps from the instruction. The planner provides an explicit roadmap.

Implementation: The planner prompt instructs the LLM to output a JSON checklist: [{"step": 1, "action": "authenticate_user", "tool": "find_user_id_by_name_zip", "args_needed": ["first_name", "last_name", "zip"]}, ...]. This checklist is passed both to the Context Injector (to narrow the tool set) and to the Completion Check (to audit after execution).

Design rationale: Unlike IRMA's Memory module which simply stores user queries, the Task Planner generates executable step decompositions. This is critical for compound tasks (e.g., "cancel all pending orders and return the watch") where the agent must perform 3–4 sequential tool calls.

4.2 Module 2: Context Injector (Code-based, Zero LLM Calls)

The Context Injector is a deterministic, rule-based module that augments the agent's system prompt based on the task checklist and domain-specific rules.

Target errors: AUTH_MISSING (422 cases), WRONG_TOOL (669 cases), NO_CONFIRMATION (276 cases).

Implementation: The module pattern-matches keywords in the user query ("return", "exchange", "cancel", "modify") and injects the corresponding domain policy sections. For example, if the planner identifies a "return" step, the injector prepends: (1) the authentication requirement ("You MUST verify the user's identity before any action"), (2) the specific tool schema for return_delivered_order_items, and (3) the confirmation requirement ("Always confirm with the user before executing irreversible actions"). It also narrows the available tool list to only the tools identified in the planner's checklist, plus read tools.

Design rationale: This is similar to IRMA's Constraints + Tool Suggestion modules, but combined into a single code-based component. By not using an LLM for this step, we avoid both latency and potential hallucination in the constraint extraction itself. The 4B model's 43.4% WRONG_TOOL rate suggests that narrowing the tool set could have an outsized impact on smaller models.

4.3 Module 3: Tool Agent (Base Qwen Model)

The Tool Agent is the unmodified Qwen3 model running in its baseline configuration (Act, ReAct, or Function Calling). It receives the augmented prompt from the Context Injector and executes tool calls against the τ-bench environment. This ensures fair comparison with Phase 1 baselines — the same model, same strategy, but with enriched input context.

4.4 Module 4: Action Verifier (Code-based, Zero LLM Calls)

The Action Verifier is the core innovation of our framework. After each agent response, it programmatically compares the agent's natural language claims against the actual tool call log.

Target errors: PREMATURE_TERMINATION (sub-pattern B: hallucinated completion, ~70% of PT cases in 32B), WRONG_ARGUMENT (798 combined argument errors).

Implementation: The verifier maintains a running log of actual tool calls made during the conversation. When the agent produces a response claiming task completion (detected by keyword matching: "cancelled", "processed", "refund", "confirmed", etc.), the verifier checks whether corresponding consequential tool calls exist in the log. If the agent claims "Your order has been cancelled" but no cancel_pending_order call was made, the verifier flags this as a hallucination and triggers a retry: the agent is prompted with "You stated the action was completed, but no tool call was recorded. Please actually execute the required tool."

Design rationale: This directly addresses the single largest failure mode in our dataset. Among 32B PREMATURE_TERMINATION samples, 7 out of 10 involved hallucinated completions with zero tool calls. This module is deliberately code-based: the comparison between claimed and actual actions is deterministic and requires no inference. The retry mechanism is bounded to a maximum of 2 retries to prevent infinite loops.

4.5 Module 5: Completion Check (Code-based, Zero LLM Calls)

The Completion Check runs once after the agent signals conversation end. It compares the Task Planner's checklist against the set of actually executed tool calls.

Target errors: PARTIAL_FULFILLMENT (40 cases), MISSING_LOOKUP (1,045 cases).

Implementation: For each step in the planner's checklist, the module verifies that the corresponding tool was called with non-trivial arguments. If steps remain incomplete, the module generates a resume prompt listing the unfinished steps and feeds it back to the Tool Agent for a second execution pass. This is bound to a single resume attempt.

Design rationale: Our analysis found that MISSING_LOOKUP errors (1,045 cases, 20.5%) frequently co-occur with argument errors — agents skip get_order_details calls and then operate on wrong order IDs. By ensuring all planned lookup steps are executed before consequential actions, this module indirectly reduces downstream argument errors.

5. Pseudocode

The following pseudocode describes the complete framework pipeline:

```
def verify_act_pipeline(user_instruction, env, model):
    # Phase 1: Pre-action
    checklist = task_planner(model, user_instruction)  # 1 LLM call
    augmented_prompt = context_injector(user_instruction, checklist, env.domain)

    # Phase 2: Execution with verification loop
    tool_log = []
    retries = 0
    MAX_RETRIES = 2
```

```
    while not conversation_ended(env):
        response = model.generate(augmented_prompt, env)  # base agent call
        new_tools = extract_tool_calls(response)
        tool_log.extend(new_tools)

        # Action Verifier (code-based)
        if claims_completion(response) and not has_matching_tools(response, tool_log):
            if retries < MAX_RETRIES:
                augmented_prompt += hallucination_correction_prompt(response,
tool_log)
                retries += 1
                continue

        env.step(response)

    # Phase 3: Completion Check (code-based)
    completed = match_checklist_to_log(checklist, tool_log)
    if completed.missing_steps and resume_attempts < 1:
        resume_prompt = generate_resume(completed.missing_steps)
        # Re-enter execution loop with resume_prompt
        ...

    return env.get_result()
```

6. Error-to-Module Mapping

The table below provides a complete mapping from each identified error type to the module designed to address it, along with the specific mechanism of intervention:

Error Type	Module	Count	Mechanism
PREMATURE_TERMINATION(never started)	Task Planner	~427	Explicit step checklist prevents agent from stalling
PREMATURE_TERMINATION(hallucinated)	Action Verifier	~640	Detects claim-vs-execution mismatch, forces retry
WRONG_TOOL	Context Injector	669	Narrows tool list to task-relevant subset
WRONG_ARGUMENT	Action Verifier	798	Validates argument values against lookup results
AUTH_MISSING	Context Injector	422	Injects mandatory auth step into prompt
MISSING_LOOKUP	Completion Check	567	Audits executed lookups against planned steps
NO_CONFIRMATION	Context Injector	276	Injects confirmation requirement into prompt
PARTIAL_FULFILLMENT	Completion Check	32	Detects incomplete checklist, resumes execution

Table 2: Complete error-to-module mapping with intervention mechanisms.

7. Computational Overhead Analysis

A key design goal of our framework is minimal computational overhead. Unlike approaches that add multiple LLM agents per conversation turn, our pipeline adds exactly one additional LLM call per task:

Module	Type	LLM Calls	When
Task Planner	LLM-powered	1 (once per task)	Conversation start
Context Injector	Code-based	0	After planning
Tool Agent	LLM-powered	N per turn (baseline)	Each turn
Action Verifier	Code-based	0	After each response
Completion Check	Code-based	0	Conversation end

Table 3: Computational overhead per module. Total additional cost: 1 LLM call per task.

For comparison, IRMA requires 3 additional LLM calls per agent turn (Memory, Constraints, and Tool Suggestion modules). On a typical τ-bench task with 10–15 agent turns, IRMA adds 30–45 LLM calls versus our 1. This is particularly significant when running on vLLM with Qwen3-32B, where each inference call consumes substantial GPU time and memory.

8. Comparison with IRMA

Dimension	IRMA	Our framework
Approach	Input reformulation (pre-action only)	Pre-action + post-action verification
Hallucination detection	Not addressed (verification-loop-free)	Action Verifier catches claim-vs-execution mismatch
LLM overhead	3 LLM calls per turn	1 LLM call per task
Error coverage	Wrong info, wrong decision, wrong argument	All 8 error types including execution hallucination
Verification	None	Deterministic tool log comparison
Task planning	Memory stores queries	Planner generates executable checklist
Training required	No	No

Table 4: Feature comparison between IRMA and our framework.

9. Expected Impact

Based on our error analysis, we estimate the following potential impact of each module on the overall failure rate:

Action Verifier targets the single largest error source: hallucinated completion in 14B/32B models. If it successfully catches even 50% of these cases and forces correct execution, this alone could improve pass^1 for 32B from ~15% to ~25%, as hallucinated PREMATURE_TERMINATION accounts for roughly 24% of all 32B failures.

Context Injector should disproportionately help the 4B model, where WRONG_TOOL accounts for 43.4% of failures. Narrowing the tool list from 14+ tools to 3–5 relevant ones reduces the selection space dramatically.

Task Planner + Completion Check should reduce PARTIAL_FULFILLMENT and the “never started” subset of PREMATURE_TERMINATION, which together account for approximately 15% of failures across models.

We anticipate the largest absolute gains on the 32B model (where hallucinated completion is the dominant failure mode) and the largest relative gains on the 4B model (where tool narrowing addresses the majority error type). The 14B model should see moderate improvements across all categories.

10. Phase 3 Implementation Plan

For Phase 3, we plan to implement the framework as a wrapper around the existing τ-bench evaluation pipeline:

1. Task Planner: A prompt template that generates JSON checklists, called once per task before the main agent loop.
2. Context Injector: A Python module with domain-specific policy dictionaries (airline/retail) and keyword-matching logic for tool narrowing.
3. Action Verifier: A Python function that hooks into the agent’s response handler, maintaining a tool call log and performing claim-vs-execution checks.
4. Completion Check: A post-conversation audit function that compares the planner’s checklist against the accumulated tool log.
5. Evaluation: Run the complete pipeline under the same τ-bench protocol as Phase 1, with identical user agent (Qwen3-32B) and the same model sizes / strategies for fair comparison.

11. Individual Contributions

Divyesh Patel: Led error analysis infrastructure. Developed automated error classification scripts (error_analysis_folder.py, error_sampler.py) for the 5-level taxonomy. Ran analysis across all 4,326 trajectories. Conducted manual review of 32B PREMATURE_TERMINATION and AUTH_MISSING samples to identify sub-patterns (hallucinated completion, never started, got lost). Produced quantitative cross-model error comparison.

Wei-An Wang: Designed framework architecture. Mapped error types to intervention modules. Researched related work (IRMA, Agent-R, CORRECT, ReIn). Authored the multi-agent proposal document including pseudocode, computational overhead analysis, and rationale connecting each module to specific error types. Developed error taxonomy visualization and architecture diagram.

Abhishek Mulasi: Contributed to error taxonomy development and trajectory sampling. Reviewed WRONG_TOOL and MISSING_LOOKUP failure patterns. Assisted with cause-oriented error grouping. Will lead Phase 3 implementation of the Context Injector and Completion Check modules.

Devesh Valluru: Contributed to error analysis and representative failure trajectory extraction. Reviewed argument error patterns (WRONG_ORDER, WRONG_ITEMS, WRONG_REPLACEMENT). Assisted with cross-model comparison plots. Will lead Phase 3 implementation of the Task Planner and Action Verifier modules.

12. References

[1] Ding et al. (2025). "IRMA: Input Reformulation for Multi-Agent Systems in τ-Bench." arXiv:2508.20931.

[2] Yao et al. (2023). "τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains." NeurIPS 2024.

[3] Yuan et al. (2025). "Agent-R: Training Language Model Agents to Reflect via Iterative Self-Training." arXiv:2501.11425.

[4] Yu et al. (2025). "CORRECT: COndensed eRror RECognition via knowledge Transfer in multi-agent systems." arXiv:2509.24088.

[5] Kim et al. (2026). "ReIn: Conversational Error Recovery with Reasoning Inception." arXiv:2602.17022.

[6] Zhu et al. (2025). "Where LLM Agents Fail and How They Can Learn From Failures." arXiv:2509.25370.

[7] Yao et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." ICLR 2023.

[8] Qin et al. (2024). "ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs." ICLR 2024.